

STM32 Ethernet Sürücüsü Mimari Tasarım ve Cache Coherency Rehberi

1. Mimarinin Genel Felsefesi

Bu doküman; STM32 serisi mikrodenetleyicilerde (F4, F7, H5, H7) çalışan esnek, yüksek performanslı ve düşük bellek izli (*zero-overhead*) bir Ethernet sürücüsünün mimari tasarımını ve karşılaşılan donanımsal kısıtların çözümünü ele almaktadır.

Sürücü mimarisi, farklı donanım jenerasyonları arasındaki farkları **çalışma zamanında (run-time)** `if/else` bloklarıyla kontrol etmek yerine, **derleme zamanında (compile-time)** çözer. Böylece kullanılmayan kodlar Flash belleğe girmez ve maksimum işlemci performansı elde edilir.

2. MAC Donanım Mimarisi: EQOS ve GMAC Farkı

STMicroelectronics, farklı işlemci ailelerinde Synopsys tarafından lisanslanan iki farklı MAC IP'si kullanır:

- GMAC (Legacy MAC):** STM32F1, F2, F4 ve F7 ailelerinde yer alır.
- EQOS (Quality of Service MAC):** STM32H5 ve H7 gibi yeni nesil ailelerde yer alır.

Temel Donanım Karşılaştırması

Donanım Özelliği	GMAC (Eski Nesil)	EQOS (Yeni Nesil)
Desteklenen Aileler	STM32F4, STM32F7	STM32H5, STM32H7
SMI (PHY Access) Register	MACMIAR	MACMDIOAR
Ring / DMA Yapısı	Descriptor'lar birbirine bağlı zincir (LinkedList) mantığındadır.	DMA'ya taban adresi ve uzunluk verilerek yönetilen Ring Buffer yapısıdır.
Gönderim Tetikleme (Tx Demand)	Poll Demand register'ına yazılır.	Tail Pointer adresi iletilir.
OWN Biti Konumu	DES0 kelimesi içerisindedir.	DES3 kelimesi içerisindedir.

Mimari Karar ve Port Dosyaları

Descriptor yapıları ve DMA yönetim mantığı kökten farklı olduğu için bu iki donanım `#ifdef` karmaşasına yol açmamak adına `eth_port_gmac.c` ve `eth_port_eqos.c` şeklinde iki ayrı dosyaya ayrılmıştır. Derleme anında sadece ilgili hedef işlemcinin port dosyası derlenir.

3. Donanım Arayüz Seçimi (VIA Register Ayrışımı)

MCU ile harici PHY çipi arasındaki RMII/MII seçiminin yapıldığı register her işlemci ailesinde farklılık gösterir (VIA kelimesi "üzerinden / kanalıyla" anlamında kullanılmıştır):

MCU Ailesi	Kullanılan MAC	Arayüz Seçim Register'ı	Açıklama
STM32F4 / F7	GMAC	SYSCFG->PMC	Klasik SYSCFG bloğu (MII_RMII_SEL)
STM32H7	EQOS	SYSCFG->PMCR	Gelişmiş SYSCFG bloğu (EPIS_SEL)
STM32H5	EQOS	SBS->PMCR	H5 ailesinde SYSCFG bloğu SBS (System Bus Support) olarak adlandırılmıştır (ETH_SEL_PHY).

Kod Uygulaması Örneği

```
ETH_Status_t ETH_Port_SelectRMII(void)
{
    #if defined(ETH_RMII_VIA_SBS)
        // STM32H5 Serisi için
        RCC->APB3ENR |= RCC_APB3ENR_SBSSEN;
        SBS->PMCR |= (4UL << SBS_PMCR_ETH_SEL_PHY_Pos);

    #elif defined(ETH_RMII_VIA_SYSCFG_PMCR)
        // STM32H7 Serisi için
        RCC->APB4ENR |= RCC_APB4ENR_SYSCFGEN;
        SYSCFG->PMCR |= (4UL << SYSCFG_PMCR_EPIS_SEL_Pos);

    #elif defined(ETH_RMII_VIA_SYSCFG_PMC)
        // STM32F4 / F7 Serisi için
        RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
        SYSCFG->PMC |= SYSCFG_PMC_MII_RMII_SEL;
    #endif

    return ETH_OK;
}
```

Not: Bu seçim, MAC clock'ları açılmadan önce yapılmalıdır. Aksi hâlde MAC birimi yanlış

4. Bellek Yönetimi ve Cache Coherency (Önbellek Tutarlılığı)

Cortex-M7 (STM32F7, H7) ve Cortex-M33 (STM32H5) çekirdeklerinde bulunan **D-Cache (Veri Önbelleği)**, DMA kullanılan sistemlerde donanımsal çakışmalara yol açabilir.

Problem: DMA – Cache Çakışması

Sistemde bellek üzerinde işlem yapan iki bağımsız aktör vardır:

1. **CPU** — Verilere hızlı erişmek için D-Cache kullanır.
2. **Ethernet DMA** — CPU'dan bağımsız olarak doğrudan SRAM'e yazar veya okur.

```
[ CPU ] <---> [ D-Cache (Hızlı) ] <---> [ Bus Matrix ] <---> [ SRAM (RAM) ]
                                     ^
[ Ethernet MAC ] <---> [ Ethernet DMA ] <---->
```

Adım Adım Oluşan Sorun

1. **Başlangıç:** Descriptor alanında `OWN = 1` yazmaktadır (DMA kullanımında). CPU bu adresi okuduğu için `OWN = 1` bilgisi D-Cache'e kopyalanır.
2. **Paket Gelmesi:** Ethernet DMA, gelen paketi SRAM'e yazar ve `OWN = 0` yapar. Ancak DMA, D-Cache'den habersizdir; bu yüzden D-Cache güncellenmez.
3. **CPU Kontrolü:** CPU `OWN` bitini okumak istediğinde SRAM yerine D-Cache'deki bayat veriyi (`OWN = 1`) okur. Sonuç olarak paketlerin gelmediğini sanır veya çöp veri okur.

5. Çözüm Yöntemi: MPU ile Non-Cacheable Alan Tanımlamak

Bu sorunu çözmenin en performanslı ve güvenilir yolu **MPU (Memory Protection Unit)** kullanarak Ethernet Descriptor ve Buffer alanlarını önbelleğe kapatmaktır.

MPU Yapılandırma Mantiği

```
+-----+
|          SRAM Bellek Haritası          |
+-----+
0x20000000 --> | Normal Veriler (D-Cache Aktif) | <-- CPU performansı yüksek
+-----+
0x30000000 --> | Ethernet Descriptors & Buffers | <-- MPU: Non-Cacheable
                | (D-Cache Devre Dışı)         | (Doğrudan SRAM okunur)
0x30008000 --> +-----+
```

Kod Düzeyinde Koşullu Derleme

Bölüm 1'de belirtilen *compile-time* çözümleme felsefesi burada da uygulanır: D-Cache barındırmayan mimarilerde fonksiyon gövdesi tamamen boşalır.

```
#if ETH_HAS_DCACHE && ETH_USE_MPU_NONCACHEABLE
    static void eth_mpu_config(void) {
        /* Descriptor ve Buffer bölgesini Non-cacheable ilan et */
    }
#else
    static void eth_mpu_config(void) {
        /* F4 gibi D-Cache olmayan mimarilerde fonksiyon boştur
        ve derleyici tarafından silinir */
    }
#endif
```

Neden Yazılımsal Invalidation Yerine MPU?

Yöntem	Değerlendirme
Software Invalidation (SCB_InvalidateDCache)	Her paket geldiğinde manuel çağrılmalıdır. 32-byte alignment zorunluluğu vardır, unutulursa sistem kilitlenir.
MPU Konfigürasyonu (önerilen)	Sadece başlangıçta (initialization) bir kez yapılır. Kodun geri kalanında geliştiricinin ekstra ön bellek temizleme işlemi yapmasına gerek kalmaz.

Dikkat: MPU programlama modeli çekirdeğe göre değişir. ARMv7-M (Cortex-M7)

RBAR / RASR ve boyut kodu kullanırken, ARMv8-M (Cortex-M33) MAIR_ATTR +

RBAR / RLAR (limit adresi) kullanır. Bu iki model birbirinin yerine kullanılamaz.

6. Özet: Derleme Zamanı Ayırışım Tablosu

Tüm mimari kararlar, hedef işlemcinin seçilmesiyle birlikte tek bir eşleme katmanında (eth_device.h) çözülür:

Hedef	MAC Ailesi	RMII Seçimi	D-Cache	MPU Gereksinimi
STM32F407	GMAC	SYSCFG->PMC	Yok	Gerekmez
STM32F767	GMAC	SYSCFG->PMC	Var	ARMv7-M MPU
STM32H563	EQOS	SBS->PMCR	Var	ARMv8-M MPU

STM32H743	EQOS	SYSCFG->PMCR	Var	ARMv7-M MPU
-----------	------	--------------	-----	-------------

Bu tablo sayesinde yeni bir MCU desteęi eklemek, sürücü çekirdeęine dokunmadan yalnızca esleme katmanına bir blok eklemekten ibarettir